```
# Step 1: Load and inspect the UCI Heart Disease dataset

%pip -q install -U ucimlrepo

import pandas as pd
from ucimlrepo import fetch_ucirepo

# Display tables more clearly in Google Colab
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

# Fetch the dataset from the official UCI ML Repository
heart_disease = fetch_ucirepo(id=45)

# Keep untouched copies of the raw features and target
X_raw = heart_disease.data.features.copy()
y_raw = heart_disease.data.targets.copy()

# 1. Dataset information
print("DATASET:", heart_disease.metadata.name)
print("UCI dataset ID:", heart_disease.metadata.uci_id)
print("Reported number of instances:",
      heart_disease.metadata.num_instances)

# 2. Dimensions
print("\nFEATURE MATRIX SHAPE:", X_raw.shape)
print("TARGET MATRIX SHAPE:", y_raw.shape)

# 3. Column names
print("\nFEATURE COLUMNS:")
print(X_raw.columns.tolist())

print("\nTARGET COLUMNS:")
print(y_raw.columns.tolist())

# 4. Preview the raw data
print("\nFIRST FIVE FEATURE ROWS:")
display(X_raw.head())

print("\nFIRST FIVE TARGET ROWS:")
display(y_raw.head())

# 5. Data types
print("\nFEATURE DATA TYPES:")
display(X_raw.dtypes.to_frame(name="Data type"))

# 6. Missing values
print("\nMISSING VALUES PER FEATURE:")
display(
    X_raw.isna()
    .sum()
    .to_frame(name="Missing values")
    .sort_values("Missing values", ascending=False)
)

# 7. Check for missing target values
print("\nMISSING TARGET VALUES:")
print(y_raw.isna().sum())

# 8. Original target distribution
print("\nRAW TARGET VALUE COUNTS:")
for column in y_raw.columns:
    print(f"\nTarget column: {column}")
    print(y_raw[column].value_counts(dropna=False).sort_index())
```

```
DATASET: Heart Disease
UCI dataset ID: 45
Reported number of instances: 303
```

```
# Step 2: Define the target and inspect the features

# A. Convert the original five-level target into a binary target.
# Keep X_raw and y_raw unchanged for reference.
y = (y_raw["num"] > 0).astype("int64")
y.name = "heart_disease"

print("BINARY TARGET DISTRIBUTION")

target_counts = y.value_counts().sort_index()

target_summary = pd.DataFrame({
    "Count": target_counts,
    "Percentage": (target_counts / len(y) * 100).round(2)
})

target_summary.index = [
    "0 - disease absent" if value == 0
    else "1 - disease present"
    for value in target_summary.index
]

display(target_summary)

# B. Inspect the numerical measurement features.
numeric_candidate_cols = [
    "age", "trestbps", "chol", "thalach", "oldpeak"
]

print("\nNUMERICAL FEATURE SUMMARY")
display(X_raw[numeric_candidate_cols].describe().T)

# C. Inspect the coded categorical/discrete features.
categorical_candidate_cols = [
    "sex", "cp", "fbs", "restecg",
    "exang", "slope", "ca", "thal"
]

print("\nCATEGORY FREQUENCIES (INCLUDING MISSING VALUES)")

for column in categorical_candidate_cols:
    print(f"\n{column}")
    display(
        X_raw[column]
        .value_counts(dropna=False)
        .sort_index()
        .to_frame(name="Count")
    )

# D. Confirm that the original features were not altered.
print("\nFINAL CHECK")
print("Feature matrix shape:", X_raw.shape)
print("Target shape:", y.shape)
print("Missing target values:", y.isna().sum())
print("Total missing feature values:", X_raw.isna().sum().sum())
```

```
MISSING VALUES PER FEATURE:
```

| | Missing values |
|---|---|
| **ca** | 4 |
| **thal** | 2 |
| **cp** | 0 |
| **sex** | 0 |
| **age** | 0 |
| **chol** | 0 |
| **trestbps** | 0 |
| **fbs** | 0 |
| **restecg** | 0 |
| **exang** | 0 |
| **thalach** | 0 |
| **slope** | 0 |

```
oldpeak               0

MISSING TARGET VALUES:
num    0
dtype: int64

RAW TARGET VALUE COUNTS:

Target column: num
num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64
```

BINARY TARGET DISTRIBUTION

1 to 2 of 2 entries Filter

Like what you see? Visit the data table notebook to learn more about interactive tables.

| index | Count | Percentage |
|---|---|---|
| 0 - disease absent | 164 | 54.13 |
| 1 - disease present | 139 | 45.87 |

Show 25 per page

Next steps: Generate code with target_summary New interactive sheet

NUMERICAL FEATURE SUMMARY

```
# Step 3: Split data, build preprocessing, and evaluate a baseline

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

# ----------------------------------------------------
# 1. Define feature groups based on the UCI metadata
# ----------------------------------------------------

numeric_features = [
    "age",
    "trestbps",
    "chol",
    "thalach",
    "oldpeak"
]

categorical_features = [
    "sex",
    "cp",
    "fbs",
    "restecg",
    "exang",
    "slope",
    "ca",
    "thal"
]

# ----------------------------------------------------
# 2. Create a stratified train/test split
# ----------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X_raw,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("DATA SPLIT")
print("Training features:", X_train.shape)
print("Testing features: ", X_test.shape)
print("Training targets:", y_train.shape)
print("Testing targets: ", y_test.shape)

print("\nCLASS DISTRIBUTION AFTER SPLITTING")

split_counts = pd.DataFrame({
    "Full dataset": y.value_counts().sort_index(),
    "Training set": y_train.value_counts().sort_index(),
    "Test set": y_test.value_counts().sort_index()
})

split_counts.index.name = "Class"
display(split_counts)

# ----------------------------------------------------
# 3. Build separate preprocessing pipelines
# ----------------------------------------------------

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

```
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_pipeline, numeric_features),
        ("categorical", categorical_pipeline, categorical_features)
    ],
    remainder="drop"
)

# ------------------------------------------------
# 4. Combine preprocessing with Logistic Regression
# ------------------------------------------------

baseline_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=2000))
])

# ------------------------------------------------
# 5. Evaluate using 5-fold cross-validation
#    on TRAINING DATA ONLY
# ------------------------------------------------

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

cv_results = cross_validate(
    estimator=baseline_pipeline,
    X=X_train,
    y=y_train,
    cv=cv,
    scoring=scoring,
    n_jobs=-1,
    error_score="raise"
)

# ------------------------------------------------
# 6. Display results for each fold
# ------------------------------------------------

metric_names = list(scoring.keys())

fold_results = pd.DataFrame({
    metric: cv_results[f"test_{metric}"]
    for metric in metric_names
})

fold_results.index = [
    f"Fold {i}" for i in range(1, len(fold_results) + 1)
]

print("\nBASELINE: LOGISTIC REGRESSION")
print("Five-fold cross-validation results:")
display(fold_results.round(3))

# ------------------------------------------------
# 7. Summarize mean and variability
# ------------------------------------------------

summary = pd.DataFrame({
    "Mean": fold_results.mean(),
    "Std": fold_results.std(ddof=1)
})

print("\nCROSS-VALIDATION SUMMARY")
display(summary.round(3))
```

```
print(
    "\nNote: These are training-set cross-validation results,",
    "not final test-set results."
)
```

```
DATA SPLIT
Training features: (242, 13)
Testing features:  (61, 13)
Training targets: (242,)
Testing targets:  (61,)

CLASS DISTRIBUTION AFTER SPLITTING
```

| Class | Full dataset | Training set | Test set |
|---|---|---|---|
| **0** | 164 | 131 | 33 |
| **1** | 139 | 111 | 28 |

```
BASELINE: LOGISTIC REGRESSION
Five-fold cross-validation results:
```

| | accuracy | precision | recall | f1 | roc_auc |
|---|---|---|---|---|---|
| **Fold 1** | 0.837 | 0.826 | 0.826 | 0.826 | 0.918 |
| **Fold 2** | 0.857 | 0.941 | 0.727 | 0.821 | 0.909 |
| **Fold 3** | 0.854 | 0.941 | 0.727 | 0.821 | 0.890 |
| **Fold 4** | 0.833 | 0.818 | 0.818 | 0.818 | 0.914 |
| **Fold 5** | 0.854 | 0.857 | 0.818 | 0.837 | 0.881 |

```
CROSS-VALIDATION SUMMARY
```

| | Mean | Std |
|---|---|---|
| **accuracy** | 0.847 | 0.011 |
| **precision** | 0.877 | 0.061 |
| **recall** | 0.783 | 0.051 |
| **f1** | 0.825 | 0.008 |
| **roc_auc** | 0.902 | 0.016 |

```
Note: These are training-set cross-validation results, not final test-set results.
```

Next steps: Generate code with `split_counts` | New interactive sheet

```
# Step 4: Compare four classifiers using identical CV folds

%pip -q install xgboost

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import cross_validate

import pandas as pd
import numpy as np

# 1. Define baseline models.
# These are starting configurations, not tuned models.

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000
    ),

    "SVM (RBF)": SVC(
        kernel="rbf",
        C=1.0
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=300,
        random_state=42,
        n_jobs=1
```

```
        ),

        "XGBoost": XGBClassifier(
            n_estimators=150,
            max_depth=2,
            learning_rate=0.03,
            subsample=0.9,
            colsample_bytree=0.9,
            reg_lambda=1.0,
            objective="binary:logistic",
            eval_metric="logloss",
            random_state=42,
            n_jobs=1
        )
    }

    # 2. Evaluate all models with the same CV folds.
    # The existing 'preprocessor', 'cv', X_train and y_train
    # come from our previous cells.

    scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc"
    }

    summary_rows = []
    fold_rows = []

    for model_name, classifier in models.items():

        pipeline = Pipeline(steps=[
            ("preprocessor", clone(preprocessor)),
            ("classifier", classifier)
        ])

        scores = cross_validate(
            estimator=pipeline,
            X=X_train,
            y=y_train,
            cv=cv,
            scoring=scoring,
            n_jobs=1,
            error_score="raise"
        )

        # Aggregate the scores over the five folds.
        row = {"Model": model_name}

        for metric in scoring:
            values = scores[f"test_{metric}"]

            row[f"{metric}_mean"] = values.mean()
            row[f"{metric}_std"] = values.std(ddof=1)

            # Preserve each fold's result for later diagnosis.
            for fold_idx, value in enumerate(values, start=1):
                fold_rows.append({
                    "Model": model_name,
                    "Fold": fold_idx,
                    "Metric": metric,
                    "Score": value
                })

        summary_rows.append(row)

    # 3. Show mean scores and variability.
    comparison = pd.DataFrame(summary_rows).set_index("Model")

    print("MODEL COMPARISON: FIVE-FOLD CROSS-VALIDATION")
    print("Mean = average score; Std = standard deviation across folds.")

    display(comparison.round(3))

    # 4. Show the individual F1 and recall scores.
    fold_results = pd.DataFrame(fold_rows)

    print("\nF1-SCORE BY FOLD")
    display(
        fold_results[fold_results["Metric"] == "f1"]
```

```
    .pivot(index="Fold", columns="Model", values="Score")
    .round(3)
)

print("\nRECALL BY FOLD")
display(
    fold_results[fold_results["Metric"] == "recall"]
    .pivot(index="Fold", columns="Model", values="Score")
    .round(3)
)

print("\nThe held-out test set has not been used.")
```

```
MODEL COMPARISON: FIVE-FOLD CROSS-VALIDATION
Mean = average score; Std = standard deviation across folds.
```

| Model | accuracy_mean | accuracy_std | precision_mean | precision_std | recall_mean | recall_std | f1_mean | f1_std | roc_auc_mea |
|---|---|---|---|---|---|---|---|---|---|
| **Logistic Regression** | 0.847 | 0.011 | 0.877 | 0.061 | 0.783 | 0.051 | 0.825 | 0.008 | 0.9( |
| **SVM (RBF)** | 0.814 | 0.026 | 0.830 | 0.083 | 0.765 | 0.089 | 0.789 | 0.030 | 0.8( |
| **Random Forest** | 0.814 | 0.026 | 0.797 | 0.030 | 0.801 | 0.090 | 0.796 | 0.041 | 0.8( |
| **XGBoost** | 0.814 | 0.027 | 0.813 | 0.056 | 0.783 | 0.100 | 0.792 | 0.038 | 0.8( |

```
F1-SCORE BY FOLD
```

| Model<br>Fold | Logistic Regression | Random Forest | SVM (RBF) | XGBoost |
|---|---|---|---|---|
| **1** | 0.826 | 0.857 | 0.833 | 0.857 |
| **2** | 0.821 | 0.750 | 0.757 | 0.769 |
| **3** | 0.821 | 0.791 | 0.800 | 0.762 |
| **4** | 0.818 | 0.809 | 0.766 | 0.792 |
| **5** | 0.837 | 0.773 | 0.791 | 0.780 |

```
RECALL BY FOLD
```

| Model<br>Fold | Logistic Regression | Random Forest | SVM (RBF) | XGBoost |
|---|---|---|---|---|
| **1** | 0.826 | 0.913 | 0.870 | 0.913 |
| **2** | 0.727 | 0.682 | 0.636 | 0.682 |
| **3** | 0.727 | 0.773 | 0.727 | 0.727 |
| **4** | 0.818 | 0.864 | 0.818 | 0.864 |
| **5** | 0.818 | 0.773 | 0.773 | 0.727 |

```
# Step 5: Hyperparameter tuning for all four classifiers

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

import pandas as pd

# 1. Metrics used to evaluate every candidate configuration.
scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

# 2. Define model configurations and controlled search spaces.
search_spaces = {
    "Logistic Regression": {
        "estimator": LogisticRegression(max_iter=2000),
```

```
        "params": {
            "classifier__C": [0.01, 0.1, 1, 10, 100],
            "classifier__class_weight": [None, "balanced"]
        }
    },

    "SVM (RBF)": {
        "estimator": SVC(kernel="rbf"),
        "params": {
            "classifier__C": [0.1, 1, 10],
            "classifier__gamma": ["scale", 0.1],
            "classifier__class_weight": [None, "balanced"]
        }
    },

    "Random Forest": {
        "estimator": RandomForestClassifier(
            n_estimators=300,
            random_state=42,
            n_jobs=1
        ),
        "params": {
            "classifier__max_depth": [None, 4, 8],
            "classifier__min_samples_leaf": [1, 2, 4]
        }
    },

    "XGBoost": {
        "estimator": XGBClassifier(
            objective="binary:logistic",
            eval_metric="logloss",
            subsample=0.9,
            colsample_bytree=0.9,
            reg_lambda=1.0,
            random_state=42,
            n_jobs=1
        ),
        "params": {
            "classifier__n_estimators": [100, 200],
            "classifier__max_depth": [2, 3],
            "classifier__learning_rate": [0.03, 0.1]
        }
    }
}

# 3. Search using the same five folds for each model.
# All preprocessing remains inside the pipeline.
searches = {}
tuning_rows = []

for model_name, config in search_spaces.items():

    pipeline = Pipeline(steps=[
        ("preprocessor", clone(preprocessor)),
        ("classifier", config["estimator"])
    ])

    search = GridSearchCV(
        estimator=pipeline,
        param_grid=config["params"],
        scoring=scoring,
        refit="roc_auc",
        cv=cv,
        n_jobs=-1,
        error_score="raise",
        return_train_score=False
    )

    print(f"\nTuning: {model_name}")

    search.fit(X_train, y_train)
    searches[model_name] = search

    # Retrieve metrics for the configuration selected by ROC-AUC.
    best_index = search.best_index_
    results = search.cv_results_

    row = {
        "Model": model_name,
        "Best parameters": search.best_params_
    }
```

```
    for metric in scoring:
        row[f"{metric}_mean"] = results[
            f"mean_test_{metric}"
        ][best_index]

        row[f"{metric}_std"] = results[
            f"std_test_{metric}"
        ][best_index]

    tuning_rows.append(row)

# 4. Present the selected configurations and their CV scores.
tuned_comparison = (
    pd.DataFrame(tuning_rows)
    .set_index("Model")
)

print("\nTUNED MODEL COMPARISON")
display(tuned_comparison.round(3))

print("\nSelected models were fitted using training data only.")
print("The held-out test set remains untouched.")
```

```
Tuning: Logistic Regression

Tuning: SVM (RBF)

Tuning: Random Forest

Tuning: XGBoost

TUNED MODEL COMPARISON
```

| Model | Best parameters | accuracy_mean | accuracy_std | precision_mean | precision_std | recall_mean | recall_std | f1_m |
|---|---|---|---|---|---|---|---|---|
| **Logistic Regression** | {'classifier__C': 0.1, 'classifier__class_weig... | 0.830 | 0.017 | 0.823 | 0.043 | 0.810 | 0.068 | 0 |
| **SVM (RBF)** | {'classifier__C': 1, 'classifier__class_weight... | 0.814 | 0.024 | 0.830 | 0.074 | 0.765 | 0.080 | 0 |
| **Random Forest** | {'classifier__max_depth': 4, 'classifier__min_... | 0.814 | 0.041 | 0.824 | 0.063 | 0.764 | 0.089 | 0 |
| **XGBoost** | {'classifier__learning_rate': | 0.834 | 0.036 | 0.855 | 0.069 | 0.783 | 0.089 | 0 |

```
# Step 6: Evaluate the selected model on the untouched test set

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    balanced_accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    PrecisionRecallDisplay
)

# ----------------------------------------------------
# 1. Retrieve the best Logistic Regression pipeline
# ----------------------------------------------------

selected_name = "Logistic Regression"
best_search = searches[selected_name]
best_model = best_search.best_estimator_

print("SELECTED MODEL:", selected_name)
print("\nBest hyperparameters:")
print(best_search.best_params_)

print("\nCross-validation ROC-AUC:",
      round(best_search.best_score_, 4))
```

```
# GridSearchCV already refitted this pipeline on all
# training records when it completed the search.

# ------------------------------------------------
# 2. Make test-set predictions exactly once
# ------------------------------------------------

y_test_pred = best_model.predict(X_test)
y_test_proba = best_model.predict_proba(X_test)[:, 1]

# ------------------------------------------------
# 3. Calculate test-set metrics
# ------------------------------------------------

tn, fp, fn, tp = confusion_matrix(
    y_test,
    y_test_pred,
    labels=[0, 1]
).ravel()

specificity = tn / (tn + fp) if (tn + fp) else 0.0

test_metrics = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall / Sensitivity",
        "Specificity",
        "F1-score",
        "Balanced Accuracy",
        "ROC-AUC",
        "Average Precision"
    ],
    "Test score": [
        accuracy_score(y_test, y_test_pred),
        precision_score(y_test, y_test_pred, zero_division=0),
        recall_score(y_test, y_test_pred, zero_division=0),
        specificity,
        f1_score(y_test, y_test_pred, zero_division=0),
        balanced_accuracy_score(y_test, y_test_pred),
        roc_auc_score(y_test, y_test_proba),
        average_precision_score(y_test, y_test_proba)
    ]
})

print("\nHELD-OUT TEST METRICS")
display(test_metrics.round(4))

# ------------------------------------------------
# 4. Confusion matrix
# ------------------------------------------------

cm = confusion_matrix(y_test, y_test_pred, labels=[0, 1])

print("\nCONFUSION MATRIX")
print(cm)

print("\nTN:", tn)
print("FP:", fp)
print("FN:", fn)
print("TP:", tp)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Disease absent (0)", "Disease present (1)"]
).plot(values_format="d")

plt.title("Logistic Regression – Held-out Test Set")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# 5. Detailed classification report
# ------------------------------------------------

print("\nCLASSIFICATION REPORT")
print(
    classification_report(
        y_test,
        y_test_pred,
        labels=[0, 1],
        target_names=["Disease absent", "Disease present"],
```

```
        digits=3,
        zero_division=0
    )
)

# ------------------------------------------------
# 6. ROC curve
# ------------------------------------------------

RocCurveDisplay.from_predictions(
    y_test,
    y_test_proba,
    name="Logistic Regression"
)

plt.title("ROC Curve — Held-out Test Set")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# 7. Precision-recall curve
# ------------------------------------------------

PrecisionRecallDisplay.from_predictions(
    y_test,
    y_test_proba,
    name="Logistic Regression"
)

plt.title("Precision-Recall Curve — Held-out Test Set")
plt.tight_layout()
plt.show()

print(
    "\nEvaluation completed. The test set has not been used "
    "to select parameters or tune the decision threshold."
)
```

```
# Step 7: Inspect feature importance using permutation

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.inspection import permutation_importance

# Use the already fitted Logistic Regression pipeline
```

```
# Use the already fitted Logistic Regression pipeline.
# This evaluates how shuffling each ORIGINAL feature
# changes test-set ROC-AUC.

importance_result = permutation_importance(
    estimator=best_model,
    X=X_test,
    y=y_test,
    scoring="roc_auc",
    n_repeats=50,
    random_state=42,
    n_jobs=-1
)

# Organize the results by original feature.
importance_df = pd.DataFrame({
    "Feature": X_test.columns,
    "Importance mean": importance_result.importances_mean,
    "Importance std": importance_result.importances_std
})

importance_df = (
    importance_df
    .sort_values("Importance mean", ascending=False)
    .reset_index(drop=True)
)

print("PERMUTATION FEATURE IMPORTANCE")
print("Metric: decrease in test ROC-AUC after shuffling a feature")
display(importance_df.round(4))

# Plot the 10 highest-ranked features.
top_features = importance_df.head(10).sort_values(
    "Importance mean",
    ascending=True
)

plt.figure(figsize=(9, 6))

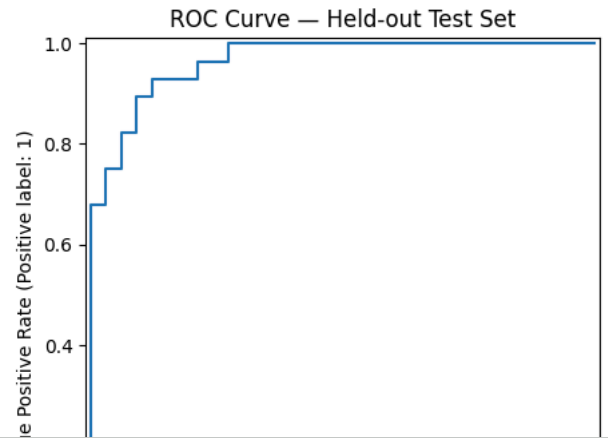
plt.barh(
    top_features["Feature"],
    top_features["Importance mean"],
    xerr=top_features["Importance std"],
    capsize=3
)

plt.axvline(0, linestyle="--", linewidth=1)
plt.xlabel("Decrease in ROC-AUC after permutation")
plt.ylabel("Original feature")
plt.title("Logistic Regression — Permutation Importance")
plt.tight_layout()
plt.show()

print(
    "\nInterpretation: larger positive values indicate that "
    "the fitted model relied more on that feature for ROC-AUC "
    "on this particular test set."
)
```

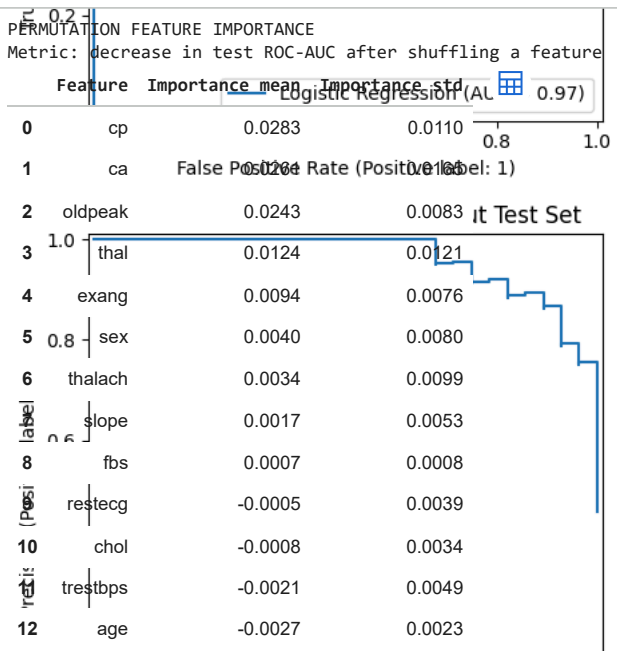
```
 Disease absent      0.933     0.848     0.889        33
Disease present      0.839     0.929     0.881        28

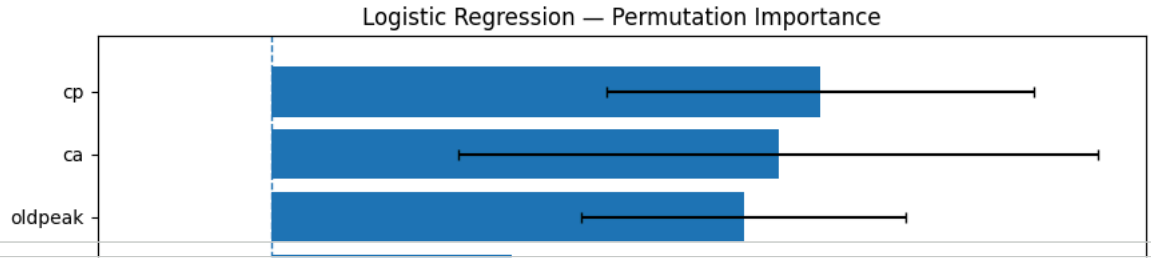
       accuracy                          0.885        61
      macro avg      0.886     0.889     0.885        61
   weighted avg      0.890     0.885     0.885        61
```



```
PERMUTATION FEATURE IMPORTANCE
Metric: decrease in test ROC-AUC after shuffling a feature
```

| | Feature | Importance mean | Importance std |
|---|---|---|---|
| 0 | cp | 0.0283 | 0.0110 |
| 1 | ca | 0.0264 | 0.0164 |
| 2 | oldpeak | 0.0243 | 0.0083 |
| 3 | thal | 0.0124 | 0.0121 |
| 4 | exang | 0.0094 | 0.0076 |
| 5 | sex | 0.0040 | 0.0080 |
| 6 | thalach | 0.0034 | 0.0099 |
| 7 | slope | 0.0017 | 0.0053 |
| 8 | fbs | 0.0007 | 0.0008 |
| 9 | restecg | -0.0005 | 0.0039 |
| 10 | chol | -0.0008 | 0.0034 |
| 11 | trestbps | -0.0021 | 0.0049 |
| 12 | age | -0.0027 | 0.0023 |




```
# Step 8: Package and verify the trained model

from pathlib import Path
import json
import sys
import zipfile
import subprocess
import importlib.metadata as package_metadata

import joblib
import numpy as np
import pandas as pd
import sklearn

# ----------------------------------------------------
# 1. Create an artifacts directory
# ----------------------------------------------------

artifact_dir = Path("/content/CodeAlpha_Disease_Prediction")
artifact_dir.mkdir(parents=True, exist_ok=True)

model_path = artifact_dir / "heart_disease_logistic_pipeline.joblib"

# Save the complete pipeline, including imputation,
# scaling, encoding and Logistic Regression.
joblib.dump(best_model, model_path)

print("Saved model:", model_path.name)

# ----------------------------------------------------
# 2. Capture environment and project metadata
# ----------------------------------------------------

packages = [
    "numpy",
    "pandas",
    "scikit-learn",
    "joblib",
    "matplotlib",
    "ucimlrepo",
    "xgboost"
]

package_versions = {}
```

```
for package in packages:
    try:
        package_versions[package] = package_metadata.version(package)
    except package_metadata.PackageNotFoundError:
        package_versions[package] = "not-installed"

test_metric_dict = {
    str(row["Metric"]): float(row["Test score"])
    for _, row in test_metrics.iterrows()
}

metadata = {
    "project_name": "CodeAlpha Disease Prediction",
    "dataset": "UCI Heart Disease",
    "uci_dataset_id": 45,
    "dataset_source": (
        "https://archive.ics.uci.edu/dataset/45/heart+disease"
    ),
    "source_target_column": "num",
    "binary_target_mapping": {
        "0": "Disease absent according to dataset target",
        "1_to_4": "Disease present according to dataset target"
    },
    "model_family": type(
        best_model.named_steps["classifier"]
    ).__name__,
    "selected_model": "Logistic Regression",
    "model_selection_metric": "Mean five-fold CV ROC-AUC",
    "cross_validation_roc_auc": float(best_search.best_score_),
    "best_parameters": best_search.best_params_,
    "feature_columns": list(X_raw.columns),
    "numeric_features": numeric_features,
    "categorical_features": categorical_features,
    "split_configuration": {
        "test_size": 0.20,
        "random_state": 42,
        "stratified": True,
        "cross_validation_folds": 5
    },
    "test_metrics": test_metric_dict,
    "package_versions": package_versions,
    "python_version": sys.version,
    "limitations": [
        "Educational project, not a clinical diagnostic tool.",
        "Performance was evaluated on one held-out split of 61 records.",
        "No independent external validation has been performed.",
        "Permutation importance is exploratory and is not causal evidence."
    ],
    "evaluation_note": (
        "The held-out split was used for final metrics and later for "
        "exploratory permutation importance. The importance results "
        "were not used to change features, model parameters or threshold."
    )
}

metadata_path = artifact_dir / "metadata.json"

with open(metadata_path, "w", encoding="utf-8") as file:
    json.dump(metadata, file, indent=2, allow_nan=False)

# ------------------------------------------------
# 3. Save evaluation results
# ------------------------------------------------

test_metrics.to_csv(
    artifact_dir / "test_metrics.csv",
    index=False
)

pd.DataFrame(
    cm,
    index=["actual_0", "actual_1"],
    columns=["predicted_0", "predicted_1"]
).to_csv(artifact_dir / "confusion_matrix.csv")

importance_df.to_csv(
    artifact_dir / "permutation_importance.csv",
    index=False
)

tuned_comparison.to_csv(
    artifact_dir / "tuned_model_comparison.csv"
```

```
)

report_dict = classification_report(
    y_test,
    y_test_pred,
    labels=[0, 1],
    target_names=["Disease absent", "Disease present"],
    output_dict=True,
    zero_division=0
)

pd.DataFrame(report_dict).T.to_csv(
    artifact_dir / "classification_report.csv"
)

# ------------------------------------------------
# 4. Create pinned dependency versions
# ------------------------------------------------

requirements_lines = [
    f"{name}=={version}"
    for name, version in package_versions.items()
    if version != "not-installed"
]

(artifact_dir / "requirements.txt").write_text(
    "\n".join(requirements_lines) + "\n",
    encoding="utf-8"
)

# ------------------------------------------------
# 5. Create a reusable prediction script
# ------------------------------------------------

predict_script = r'''
import argparse
import json
from pathlib import Path

import joblib
import pandas as pd

HERE = Path(__file__).resolve().parent

MODEL_PATH = HERE / "heart_disease_logistic_pipeline.joblib"
METADATA_PATH = HERE / "metadata.json"


def main():
    parser = argparse.ArgumentParser(
        description="Run the educational heart-disease classifier."
    )
    parser.add_argument(
        "--input",
        required=True,
        help="CSV file containing the original model input features."
    )
    parser.add_argument(
        "--output",
        default="predictions.csv",
        help="Destination CSV for predictions."
    )
    args = parser.parse_args()

    with open(METADATA_PATH, "r", encoding="utf-8") as file:
        metadata = json.load(file)

    feature_columns = metadata["feature_columns"]
    input_data = pd.read_csv(args.input)

    missing_columns = [
        column for column in feature_columns
        if column not in input_data.columns
    ]

    if missing_columns:
        parser.error(
            f"Input CSV is missing required columns: {missing_columns}"
        )

    # Preserve the training-time feature order.
    X = input_data.loc[:, feature_columns]
```

```
    model = joblib.load(MODEL_PATH)

    predictions = model.predict(X)
    probabilities = model.predict_proba(X)[:, 1]

    output = input_data.copy()
    output["predicted_label"] = predictions
    output["class_1_model_score"] = probabilities

    output.to_csv(args.output, index=False)

    print(f"Rows processed: {len(output)}")
    print(f"Predictions saved to: {args.output}")
    print("These outputs are educational model estimates, not diagnoses.")


if __name__ == "__main__":
    main()
'''

(artifact_dir / "predict.py").write_text(
    predict_script.strip() + "\n",
    encoding="utf-8"
)

# ------------------------------------------------
# 6. Create a README
# ------------------------------------------------

readme = f"""# CodeAlpha — Disease Prediction

## Project overview

An educational binary-classification project using the UCI Heart Disease
dataset. The selected model is Logistic Regression, chosen using mean
five-fold cross-validation ROC-AUC on the training split.

## Dataset

Official source:
https://archive.ics.uci.edu/dataset/45/heart+disease

The original target `num` is mapped to a binary target:
- 0 -> 0: disease absent according to the dataset target
- 1, 2, 3, 4 -> 1: disease present according to the dataset target

## Held-out evaluation

The 20% stratified test split contains 61 records.

- Accuracy: {test_metric_dict["Accuracy"]:.4f}
- Precision: {test_metric_dict["Precision"]:.4f}
- Recall / Sensitivity: {test_metric_dict["Recall / Sensitivity"]:.4f}
- Specificity: {test_metric_dict["Specificity"]:.4f}
- F1-score: {test_metric_dict["F1-score"]:.4f}
- ROC-AUC: {test_metric_dict["ROC-AUC"]:.4f}

See `test_metrics.csv`, `confusion_matrix.csv` and
`classification_report.csv` for additional results.

## Running predictions

Install the packages listed in `requirements.txt`. Place an input CSV
with all 13 required feature columns in the project directory, then run:

```bash
python predict.py --input your_features.csv --output predictions.csv
```

The pipeline handles missing feature values through its fitted
preprocessing steps. The input CSV must still contain every required column.

## Files

- `heart_disease_logistic_pipeline.joblib`: full fitted pipeline
- `metadata.json`: model configuration, feature schema and limitations
- `predict.py`: reusable command-line prediction script
- `requirements.txt`: package versions
- `test_metrics.csv`: held-out evaluation metrics
- `confusion_matrix.csv`: confusion-matrix counts
- `classification_report.csv`: class-wise evaluation
```

```
- `permutation_importance.csv`: exploratory feature importance
- `tuned_model_comparison.csv`: cross-validation model comparison

## Limitations

This project is for machine-learning education only. It is not a medical
diagnostic tool. Its test results come from a single 61-record split of
a small historical dataset, and no independent external validation has
been performed.

Feature importance is exploratory, not causal. The hold-out data was used
for evaluation and subsequent exploratory importance analysis; no
parameter, feature-selection or threshold decisions were made from that
importance analysis. The reported metrics do not establish clinical
validity or generalization to other populations.
"""

(artifact_dir / "README.md").write_text(
    readme,
    encoding="utf-8"
)

# ------------------------------------------------
# 7. Verify that the model reloads correctly
# ------------------------------------------------

reloaded_model = joblib.load(model_path)

# Use a training record only for this persistence smoke test.
smoke_input = X_train.iloc[[0]].copy()

original_prediction = best_model.predict(smoke_input)
reloaded_prediction = reloaded_model.predict(smoke_input)

original_probability = best_model.predict_proba(smoke_input)
reloaded_probability = reloaded_model.predict_proba(smoke_input)

np.testing.assert_array_equal(
    original_prediction,
    reloaded_prediction
)

np.testing.assert_allclose(
    original_probability,
    reloaded_probability,
    rtol=0,
    atol=1e-12
)

print("\nMODEL RELOAD CHECK: PASSED")
print("Predictions match before and after saving.")
print("Predicted label:", int(reloaded_prediction[0]))
print(
    "Estimated class-1 score:",
    round(float(reloaded_probability[0, 1]), 4)
)

# ------------------------------------------------
# 8. Create a ZIP archive of the project artifacts
# ------------------------------------------------

zip_path = Path("/content/CodeAlpha_Disease_Prediction_Artifacts.zip")

with zipfile.ZipFile(
    zip_path,
    "w",
    compression=zipfile.ZIP_DEFLATED
) as archive:
    for file_path in artifact_dir.rglob("*"):
        if file_path.is_file():
            archive.write(
                file_path,
                arcname=(
                    artifact_dir.name + "/"
                    + str(file_path.relative_to(artifact_dir))
                )
            )

print("\nPROJECT ARTIFACTS CREATED")
for file_path in sorted(artifact_dir.iterdir()):
    if file_path.is_file():
        print(f"{file_path.name}: {file_path.stat().st_size:,} bytes")
```

```
print(f"\nZIP ARCHIVE: {zip_path}")
print(f"ZIP SIZE: {zip_path.stat().st_size:,} bytes")
```

```
Saved model: heart_disease_logistic_pipeline.joblib

MODEL RELOAD CHECK: PASSED
Predictions match before and after saving.
Predicted label: 1
Estimated class-1 score: 0.661

PROJECT ARTIFACTS CREATED
README.md: 2,326 bytes
classification_report.csv: 421 bytes
confusion_matrix.csv: 53 bytes
heart_disease_logistic_pipeline.joblib: 4,970 bytes
metadata.json: 2,322 bytes
permutation_importance.csv: 666 bytes
predict.py: 1,696 bytes
requirements.txt: 112 bytes
test_metrics.csv: 275 bytes
tuned_model_comparison.csv: 1,295 bytes

ZIP ARCHIVE: /content/CodeAlpha_Disease_Prediction_Artifacts.zip
ZIP SIZE: 8,565 bytes
```

```
# Step 9: End-to-end test of the packaged prediction script

import sys
import tempfile
import subprocess
from pathlib import Path

import numpy as np
import pandas as pd

# Use five training records for a smoke test.
# These rows will not be used for model training or tuning.
smoke_input = X_train.iloc[:5].copy()

script_path = artifact_dir / "predict.py"
model_path = artifact_dir / "heart_disease_logistic_pipeline.joblib"
metadata_path = artifact_dir / "metadata.json"

assert script_path.exists(), "Prediction script is missing."
assert model_path.exists(), "Saved model is missing."
assert metadata_path.exists(), "Metadata file is missing."

with tempfile.TemporaryDirectory() as temp_dir:
    temp_dir = Path(temp_dir)

    input_path = temp_dir / "smoke_input.csv"
    output_path = temp_dir / "smoke_output.csv"

    # Deliberately reverse the columns to test column alignment.
    smoke_input.iloc[:, ::-1].to_csv(input_path, index=False)

    # Run the actual standalone script as a separate process.
    command = [
        sys.executable,
        str(script_path),
        "--input", str(input_path),
        "--output", str(output_path)
    ]

    result = subprocess.run(
        command,
        capture_output=True,
        text=True,
        timeout=120
    )

    print("SCRIPT RETURN CODE:", result.returncode)
    print("\nSCRIPT STANDARD OUTPUT:")
    print(result.stdout)

    if result.stderr:
        print("\nSCRIPT STANDARD ERROR:")
        print(result.stderr)

    if result.returncode != 0:
        raise RuntimeError("The standalone prediction script failed.")
```

```
    assert output_path.exists(), "Prediction CSV was not created."

    # Load the output written by predict.py.
    script_output = pd.read_csv(output_path)

    # Calculate expected values directly from the fitted pipeline.
    expected_labels = best_model.predict(smoke_input)
    expected_scores = best_model.predict_proba(smoke_input)[:, 1]

    print("\nPREDICTION OUTPUT")
    display(
        script_output[
            ["predicted_label", "class_1_model_score"]
        ].round(4)
    )

    # Verify result count and prediction consistency.
    assert len(script_output) == len(smoke_input)

    np.testing.assert_array_equal(
        script_output["predicted_label"].to_numpy(),
        expected_labels
    )

    np.testing.assert_allclose(
        script_output["class_1_model_score"].to_numpy(),
        expected_scores,
        rtol=0,
        atol=1e-12
    )

    print("\nEND-TO-END CHECK: PASSED")
    print("Correct number of rows: 5")
    print("Column alignment: passed")
    print("Predicted labels match: passed")
    print("Class-1 scores match: passed")

print("\nNo model retraining or threshold tuning was performed.")
```

```
SCRIPT RETURN CODE: 0

SCRIPT STANDARD OUTPUT:
Rows processed: 5
Predictions saved to: /tmp/tmpdv247yi1/smoke_output.csv
These outputs are educational model estimates, not diagnoses.


PREDICTION OUTPUT
```

| | predicted_label | class_1_model_score |
|---|---|---|
| **0** | 1 | 0.6610 |
| **1** | 0 | 0.1625 |
| **2** | 0 | 0.3363 |
| **3** | 0 | 0.3132 |
| **4** | 1 | 0.6906 |

```
END-TO-END CHECK: PASSED
Correct number of rows: 5
Column alignment: passed
Predicted labels match: passed
Class-1 scores match: passed

No model retraining or threshold tuning was performed.
```

```
# Step 10: Finalize the GitHub-ready project

from pathlib import Path
import json
import zipfile
import subprocess
import sys

import pandas as pd
import numpy as np

from google.colab import files
```

```
# ------------------------------------------------
# 1. Prepare project directories
# ------------------------------------------------

repo_dir = Path("/content/CodeAlpha_Disease_Prediction")
examples_dir = repo_dir / "examples"
examples_dir.mkdir(parents=True, exist_ok=True)

# ------------------------------------------------
# 2. Create a sample input CSV
# ------------------------------------------------

# One example from the public UCI dataset.
# This is a software smoke-test example, not a new test
# evaluation and not a clinical recommendation.
sample_input = X_raw.iloc[[0]].copy()
sample_path = examples_dir / "sample_input.csv"
sample_input.to_csv(sample_path, index=False)

# ------------------------------------------------
# 3. Validate the bundled prediction script
# ------------------------------------------------

model_path = repo_dir / "heart_disease_logistic_pipeline.joblib"
predict_path = repo_dir / "predict.py"
metadata_path = repo_dir / "metadata.json"

for path in [model_path, predict_path, metadata_path]:
    assert path.exists(), f"Required project file missing: {path.name}"

sample_predictions_path = examples_dir / "sample_predictions.csv"

result = subprocess.run(
    [
        sys.executable,
        str(predict_path),
        "--input", str(sample_path),
        "--output", str(sample_predictions_path)
    ],
    capture_output=True,
    text=True,
    timeout=120
)

print("SAMPLE PREDICTION SCRIPT RETURN CODE:", result.returncode)
print(result.stdout)

if result.stderr:
    print("SCRIPT ERROR OUTPUT:")
    print(result.stderr)

if result.returncode != 0:
    raise RuntimeError("Prediction-script validation failed.")

sample_output = pd.read_csv(sample_predictions_path)

expected_label = int(best_model.predict(sample_input)[0])
expected_score = float(best_model.predict_proba(sample_input)[0, 1])

assert len(sample_output) == 1
assert int(sample_output.loc[0, "predicted_label"]) == expected_label
assert np.isclose(
    float(sample_output.loc[0, "class_1_model_score"]),
    expected_score,
    rtol=0,
    atol=1e-12
)

print("SAMPLE CSV VALIDATION: PASSED")

# ------------------------------------------------
# 4. Create a complete README
# ------------------------------------------------

cv_table = tuned_comparison[
    [
        "accuracy_mean",
        "precision_mean",
        "recall_mean",
        "f1_mean",
        "roc_auc_mean"
    ]
```

```
].round(3)

cv_table.index.name = "Model"
cv_markdown = cv_table.to_markdown()

test_table = test_metrics.to_markdown(index=False)

readme = f"""# CodeAlpha — Disease Prediction

An educational binary-classification project using the UCI Heart
Disease dataset and scikit-learn.

> **Important:** This is an educational machine-learning project,
> not a medical diagnostic tool. Its reported test performance comes
> from one 61-record hold-out split. It has not undergone independent
> external clinical validation.

## 1. Objective

Predict the dataset-defined presence or absence of heart disease
from 13 structured input features.

Dataset: [UCI Heart Disease](https://archive.ics.uci.edu/dataset/45/heart+disease)

The commonly used Cleveland dataset contains 303 records and 13
features. The original `num` target has values from 0 to 4. For this
project, it is converted into a binary outcome:

- `0` -> 0: disease absent according to the dataset target
- `1`, `2`, `3`, `4` -> 1: disease present according to the dataset target

This is a benchmark classification convention, not a universal
clinical definition.

## 2. Models compared

Four classifier families were compared:

- Logistic Regression
- Support Vector Machine with an RBF kernel
- Random Forest
- XGBoost

The models were evaluated using the same stratified five-fold
cross-validation scheme on the training split. Hyperparameters were
selected by mean cross-validation ROC-AUC.

### Cross-validation comparison

{cv_markdown}

The selected candidate was Logistic Regression, which had the highest
mean cross-validation ROC-AUC and F1-score in this experiment.
This is a development-stage selection, not evidence that the model
is best for other populations.

## 3. Data preparation

The workflow includes:

1. Inspection of the raw data, data types, target values and missingness.
2. Binary target construction.
3. Stratified train/test split using `test_size=0.20` and `random_state=42`.
4. Median imputation and standardization for numerical features.
5. Most-frequent imputation and one-hot encoding for categorical features.
6. Preprocessing inside a scikit-learn pipeline to reduce data leakage.
7. Cross-validation-based model comparison and hyperparameter tuning.
8. Final evaluation on a held-out set.
9. Exploratory permutation feature importance.

The held-out set contains 61 records. It was not used to select
hyperparameters or tune the decision threshold. After the final
evaluation, it was also used for exploratory permutation importance;
that analysis was not used to alter the model.

## 4. Held-out test results

{test_table}

Confusion matrix:

| Actual class | Predicted 0 | Predicted 1 |
```

```
|---|---:|---:|
| 0 — disease absent | {int(cm[0, 0])} | {int(cm[0, 1])} |
| 1 — disease present | {int(cm[1, 0])} | {int(cm[1, 1])} |

The model correctly classified {int((y_test.to_numpy() == y_test_pred).sum())}
of 61 records in this particular test split. The small evaluation sample
means the metrics are uncertain and may vary on new data.

## 5. Project structure

- `heart_disease_logistic_pipeline.joblib` — fitted preprocessing and model pipeline
- `predict.py` — command-line prediction script
- `metadata.json` — model configuration, data mapping, versions and limitations
- `requirements.txt` — dependency versions from the training environment
- `test_metrics.csv` — held-out test metrics
- `confusion_matrix.csv` — held-out confusion-matrix counts
- `classification_report.csv` — class-wise classification metrics
- `tuned_model_comparison.csv` — cross-validation model comparison
- `permutation_importance.csv` — exploratory feature-importance output
- `examples/sample_input.csv` — one illustrative input row from the UCI dataset
- `examples/sample_predictions.csv` — prediction-script smoke-test output
- `MODEL_CARD.md` — model intended use and limitations
- `REFERENCES.md` — source documentation and research reporting guidance

The exported Google Colab notebook should also be included in the
GitHub repository to preserve the complete interactive workflow.

## 6. Run a prediction

Install the listed dependencies in a compatible Python environment:

```bash
pip install -r requirements.txt
```

Then run:

```bash
python predict.py --input examples/sample_input.csv --output predictions.csv
```

The input CSV must contain all 13 expected feature columns.
Column order does not matter because the script aligns columns to
the recorded feature schema.

The output includes:

- `predicted_label`: binary classification at the model's default threshold
- `class_1_model_score`: model-estimated probability for class 1

The probability is not a validated clinical risk estimate. Do not use
these outputs to diagnose or make treatment decisions.

## 7. Reproducibility

The metadata records the dataset source, feature schema, target mapping,
split configuration, selected parameters, test metrics and installed
package versions.

For `joblib` compatibility, use the recorded dependency versions.
Only load a serialized model from a trusted source.

## 8. Limitations

- The dataset is small and historical.
- Only one held-out split was evaluated.
- No external validation or prospective study was performed.
- Test metrics may be sensitive to the individual observations in the split.
- The selected model's results do not demonstrate clinical utility or
  generalization to other hospitals or populations.
- Permutation importance measures model reliance, not causality.
- The dataset's diagnostic target is not a substitute for medical assessment.

## 9. References

1. UCI Machine Learning Repository. Heart Disease dataset.
   https://archive.ics.uci.edu/dataset/45/heart+disease
2. Scikit-learn. Pipelines and composite estimators.
   https://scikit-learn.org/stable/modules/compose.html
3. Scikit-learn. Cross-validation.
   https://scikit-learn.org/stable/modules/cross_validation.html
4. Scikit-learn. Permutation feature importance.
```

```
   https://scikit-learn.org/stable/modules/permutation_importance.html
5. Scikit-learn. Model persistence.
   https://scikit-learn.org/stable/model_persistence.html
6. Collins GS, Moons KGM, Dhiman P, et al. TRIPOD+AI statement:
   updated guidance for reporting clinical prediction models that use
   regression or machine learning methods. BMJ. 2024;385:e078378.
   https://doi.org/10.1136/bmj-2023-078378
"""

(repo_dir / "README.md").write_text(readme, encoding="utf-8")

# ------------------------------------------------
# 5. Add a model card
# ------------------------------------------------

model_card = f"""# Model Card: Heart Disease Classification

## Model

Logistic Regression with a fitted scikit-learn preprocessing pipeline.

## Intended use

Educational demonstration of tabular binary classification,
preprocessing, cross-validation, evaluation and model packaging.

## Out-of-scope use

Clinical diagnosis, triage, treatment planning, population screening,
or any patient-care decision.

## Data

Source: UCI Heart Disease dataset (ID 45), commonly used Cleveland setup.
Original target values 1-4 were grouped into class 1; value 0 became class 0.

## Evaluation setup

- Training split: 242 records
- Held-out test split: 61 records
- Stratified split, random state 42
- Five-fold stratified cross-validation used during model selection
- Default classification threshold: 0.5

## Held-out results

{test_table}

Confusion matrix: TN={int(cm[0,0])}, FP={int(cm[0,1])},
FN={int(cm[1,0])}, TP={int(cm[1,1])}.

## Key limitations

The dataset is small and historical. The evaluation uses one test split,
without independent external validation. The test results should not be
interpreted as reliable estimates of performance in clinical practice.
Model probabilities have not been established as calibrated clinical risks.

Permutation feature importance is exploratory and does not demonstrate
causality or clinical importance.
"""

(repo_dir / "MODEL_CARD.md").write_text(
    model_card, encoding="utf-8"
)

# ------------------------------------------------
# 6. Add references and attribution
# ------------------------------------------------

references = """# References

## Dataset

UCI Machine Learning Repository. Heart Disease.
https://archive.ics.uci.edu/dataset/45/heart+disease

The UCI page identifies the dataset's classification task, feature
descriptions, target coding, and historical source. Review the dataset
page for its citation and licence information.

## Software documentation
```

```
- Scikit-learn pipelines:
  https://scikit-learn.org/stable/modules/compose.html
- Scikit-learn cross-validation:
  https://scikit-learn.org/stable/modules/cross_validation.html
- Scikit-learn permutation importance:
  https://scikit-learn.org/stable/modules/permutation_importance.html
- Scikit-learn model persistence and security:
  https://scikit-learn.org/stable/model_persistence.html
- XGBoost documentation:
  https://xgboost.readthedocs.io/en/latest/get_started.html
- GitHub README guidance:
  https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about

## Research reporting guidance

Collins GS, Moons KGM, Dhiman P, et al. TRIPOD+AI statement: updated
guidance for reporting clinical prediction models that use regression
or machine learning methods. BMJ. 2024;385:e078378.
https://doi.org/10.1136/bmj-2023-078378

This reporting guideline informs the emphasis on transparent methods,
explicit limitations, and clear separation between development and
evaluation. It does not validate this project's model.
"""

(repo_dir / "REFERENCES.md").write_text(
    references, encoding="utf-8"
)

# ------------------------------------------------
# 7. Add a citation metadata file
# ------------------------------------------------

citation = """cff-version: 1.2.0
message: "Please cite the UCI Heart Disease dataset and TRIPOD+AI guidance when appropriate."
title: "CodeAlpha Disease Prediction: Heart Disease Classification"
version: 1.0.0
date-released: 2026-10-19
"""

(repo_dir / "CITATION.cff").write_text(
    citation, encoding="utf-8"
)

# ------------------------------------------------
# 8. Add a safe, minimal .gitignore
# ------------------------------------------------

gitignore = """__pycache__/
*.py[cod]
.ipynb_checkpoints/
.venv/
venv/
.env
.DS_Store
"""

(repo_dir / ".gitignore").write_text(
    gitignore, encoding="utf-8"
)

# ------------------------------------------------
# 9. Create the final repository ZIP
# ------------------------------------------------

zip_path = Path(
    "/content/CodeAlpha_Disease_Prediction_Repository.zip"
)

with zipfile.ZipFile(
    zip_path,
    "w",
    compression=zipfile.ZIP_DEFLATED
) as archive:
    for file_path in sorted(repo_dir.rglob("*")):
        if file_path.is_file():
            archive.write(
                file_path,
                arcname=(
                    repo_dir.name + "/"
                    + str(file_path.relative_to(repo_dir))
```

```
                )
            )

# ------------------------------------------------
# 10. Final validation and inventory
# ------------------------------------------------

with zipfile.ZipFile(zip_path, "r") as archive:
    bad_member = archive.testzip()
    assert bad_member is None, (
        f"ZIP integrity check failed at: {bad_member}"
    )

print("\nFINAL PROJECT FILES")
for file_path in sorted(repo_dir.rglob("*")):
    if file_path.is_file():
        print(
            f"{file_path.relative_to(repo_dir)} "
            f"({file_path.stat().st_size:,} bytes)"
        )

print("\nZIP INTEGRITY: PASSED")
print(f"ARCHIVE: {zip_path}")
print(f"ARCHIVE SIZE: {zip_path.stat().st_size:,} bytes")

# Trigger download to your computer.
files.download(str(zip_path))
```

```
SAMPLE PREDICTION SCRIPT RETURN CODE: 0
Rows processed: 1
Predictions saved to: /content/CodeAlpha_Disease_Prediction/examples/sample_predictions.csv
These outputs are educational model estimates, not diagnoses.

SAMPLE CSV VALIDATION: PASSED

FINAL PROJECT FILES
.gitignore (71 bytes)
CITATION.cff (222 bytes)
MODEL_CARD.md (1,668 bytes)
README.md (7,152 bytes)
REFERENCES.md (1,432 bytes)
classification_report.csv (421 bytes)
confusion_matrix.csv (53 bytes)
examples/sample_input.csv (112 bytes)
examples/sample_predictions.csv (169 bytes)
heart_disease_logistic_pipeline.joblib (4,970 bytes)
metadata.json (2,322 bytes)
permutation_importance.csv (666 bytes)
predict.py (1,696 bytes)
requirements.txt (112 bytes)
test_metrics.csv (275 bytes)
tuned_model_comparison.csv (1,295 bytes)

ZIP INTEGRITY: PASSED
ARCHIVE: /content/CodeAlpha_Disease_Prediction_Repository.zip
ARCHIVE SIZE: 13,363 bytes
```

```
from pathlib import Path
import shutil
import subprocess
import sys

from google.colab import files

# Existing project created in our earlier steps
repo_dir = Path("/content/CodeAlpha_Disease_Prediction")
models_dir = repo_dir / "models"
examples_dir = repo_dir / "examples"
outputs_dir = repo_dir / "outputs"

models_dir.mkdir(parents=True, exist_ok=True)
outputs_dir.mkdir(parents=True, exist_ok=True)

# 1. Upload the corrected predict.py downloaded from this chat
uploaded = files.upload()

if "predict.py" not in uploaded:
    raise FileNotFoundError(
        "Please upload the downloaded file with the exact name predict.py."
    )

# Replace the old predictor
```

```
shutil.copy2(
    Path("/content/predict.py"),
    repo_dir / "predict.py"
)

# 2. Move the model from the old root location into models/
old_model_path = repo_dir / "heart_disease_logistic_pipeline.joblib"
new_model_path = models_dir / "heart_disease_logistic_pipeline.joblib"

if old_model_path.exists() and not new_model_path.exists():
    shutil.move(str(old_model_path), str(new_model_path))
elif new_model_path.exists():
    print("Model already exists in models/.")
else:
    raise FileNotFoundError(
        "The saved model was not found. Expected it at:\n"
        f"{old_model_path}\nor\n{new_model_path}"
    )

# 3. Check all required files
required_files = [
    repo_dir / "predict.py",
    new_model_path,
    repo_dir / "metadata.json",
    repo_dir / "requirements.txt",
    examples_dir / "sample_input.csv"
]

print("\nPROJECT FILE CHECK")
for path in required_files:
    status = "FOUND" if path.is_file() else "MISSING"
    print(f"{status}: {path.relative_to(repo_dir)}")

if not all(path.is_file() for path in required_files):
    raise FileNotFoundError(
        "Some required files are missing. Resolve these before proceeding."
    )

# 4. Run the actual command-line script end to end
sample_input = examples_dir / "sample_input.csv"

result = subprocess.run(
    [
        sys.executable,
        str(repo_dir / "predict.py"),
        "--input",
        str(sample_input)
    ],
    capture_output=True,
    text=True,
    timeout=120
)

print("\nPREDICTOR OUTPUT")
print(result.stdout)

if result.stderr:
    print("\nERROR OUTPUT")
    print(result.stderr)

if result.returncode != 0:
    raise RuntimeError("The updated prediction script failed.")

prediction_file = outputs_dir / "predictions.csv"

if not prediction_file.is_file():
    raise FileNotFoundError(
        "The script ran but did not create outputs/predictions.csv."
    )

print("\nPREDICTION FILE")
display(__import__("pandas").read_csv(prediction_file))

print("\nPORTABILITY SMOKE TEST: PASSED")
```

Choose files predict.py

- **predict.py**(text/x-python) - 7980 bytes, last modified: 19/10/2026 - 100% done

```
Saving predict.py to predict.py

PROJECT FILE CHECK
FOUND: predict.py
FOUND: models/heart_disease_logistic_pipeline.joblib
FOUND: metadata.json
FOUND: requirements.txt
FOUND: examples/sample_input.csv

PREDICTOR OUTPUT

Prediction completed successfully.
Input file: /content/CodeAlpha_Disease_Prediction/examples/sample_input.csv
Rows processed: 1
Predictions saved to: /content/CodeAlpha_Disease_Prediction/outputs/predictions.csv
Labels represent the dataset's binary target mapping.
Class-1 scores are model outputs, not calibrated clinical risks or diagnoses.


PREDICTION FILE
```

| | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | predicted_label | predicted_class | cl |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 63 | 1 | 1 | 145 | 233 | 1 | 2 | 150 | 0 | 2.3 | 3 | 0.0 | 6.0 | 0 | Disease absent according to dataset target | |

```
# Step 10: Update documentation and rebuild the final ZIP

from pathlib import Path
import json
import zipfile
import pandas as pd

from google.colab import files

repo_dir = Path("/content/CodeAlpha_Disease_Prediction")
models_dir = repo_dir / "models"
examples_dir = repo_dir / "examples"
outputs_dir = repo_dir / "outputs"

models_dir.mkdir(parents=True, exist_ok=True)
examples_dir.mkdir(parents=True, exist_ok=True)
outputs_dir.mkdir(parents=True, exist_ok=True)

# ------------------------------------------------
# 1. Verify the current project structure
# ------------------------------------------------

required_files = [
    repo_dir / "predict.py",
    models_dir / "heart_disease_logistic_pipeline.joblib",
    repo_dir / "metadata.json",
    repo_dir / "requirements.txt",
    examples_dir / "sample_input.csv",
    repo_dir / "MODEL_CARD.md",
    repo_dir / "REFERENCES.md"
]

missing_files = [
    str(path.relative_to(repo_dir))
    for path in required_files
    if not path.is_file()
]

if missing_files:
    raise FileNotFoundError(
        "Required files are missing:\n" + "\n".join(missing_files)
    )

with open(repo_dir / "metadata.json", encoding="utf-8") as f:
    metadata = json.load(f)

# ------------------------------------------------
# 2. Read the saved evaluation results
# ------------------------------------------------

test_metrics = pd.read_csv(repo_dir / "test_metrics.csv")
test_metric_values = {
    str(row["Metric"]): float(row["Test score"])
    for _, row in test_metrics.iterrows()
```

```
}

cm_df = pd.read_csv(repo_dir / "confusion_matrix.csv", index_col=0)

tn = int(cm_df.loc["actual_0", "predicted_0"])
fp = int(cm_df.loc["actual_0", "predicted_1"])
fn = int(cm_df.loc["actual_1", "predicted_0"])
tp = int(cm_df.loc["actual_1", "predicted_1"])

comparison = pd.read_csv(
    repo_dir / "tuned_model_comparison.csv",
    index_col=0
)

# Format the comparison as Markdown without requiring tabulate.
metric_columns = [
    ("accuracy_mean", "Accuracy"),
    ("precision_mean", "Precision"),
    ("recall_mean", "Recall"),
    ("f1_mean", "F1"),
    ("roc_auc_mean", "ROC-AUC")
]

cv_lines = [
    "| Model | " + " | ".join(name for _, name in metric_columns) + " |",
    "|---|" + "---:|" * len(metric_columns)
]

for model_name, row in comparison.iterrows():
    values = [
        f"{float(row[column]):.3f}"
        for column, _ in metric_columns
    ]
    cv_lines.append(
        f"| {model_name} | " + " | ".join(values) + " |"
    )

cv_markdown = "\n".join(cv_lines)

test_lines = [
    "| Metric | Test score |",
    "|---|---:|"
]

for _, row in test_metrics.iterrows():
    test_lines.append(
        f"| {row['Metric']} | {float(row['Test score']):.4f} |"
    )

test_markdown = "\n".join(test_lines)

# ------------------------------------------------
# 3. Replace README with paths that match the project
# ------------------------------------------------

readme = f"""# CodeAlpha — Disease Prediction

An educational machine-learning project for binary classification
using the UCI Heart Disease dataset.

> **Important:** This model is for educational use only. It is not a
> medical diagnostic tool. Its reported performance comes from one
> 61-record held-out split and has not been independently validated
> on an external clinical dataset.

## Objective

Predict the dataset-defined presence or absence of heart disease from
13 structured input features.

**Dataset:** [UCI Heart Disease](https://archive.ics.uci.edu/dataset/45/heart+disease)

The original target column `num` contains values 0 through 4. This
project maps value 0 to class 0 and values 1 through 4 to class 1.

This binary mapping is a common benchmark convention; it is not a
general clinical diagnosis or an estimate of future disease risk.

## Workflow

1. Inspect the raw dataset, feature types, missing values and target.
2. Prepare the binary target.
3. Split the data using a stratified 80/20 train/test split
```

```
   (`random_state=42`).
4. Impute and standardize numerical features.
5. Impute and one-hot encode categorical features.
6. Compare Logistic Regression, SVM, Random Forest and XGBoost.
7. Tune candidate models using five-fold cross-validation on the
   training split.
8. Evaluate the selected Logistic Regression model on the held-out test set.
9. Examine exploratory permutation feature importance.
10. Save, reload and smoke-test the prediction pipeline.

Preprocessing is kept inside the scikit-learn pipeline to avoid
fitting learned preprocessing on held-out evaluation data.

## Cross-validation comparison

The table shows mean scores for the selected configuration of each
model, measured across five folds of the training data.

{cv_markdown}

Logistic Regression was selected using mean cross-validation ROC-AUC.
These cross-validation metrics were used during model selection and
should not be confused with the held-out test results below.

## Held-out test results

{test_markdown}

### Confusion matrix

| Actual class | Predicted 0 | Predicted 1 |
|---|---:|---:|
| Disease absent (0) | {tn} | {fp} |
| Disease present (1) | {fn} | {tp} |

The test split contains {tn + fp + fn + tp} records. These results
describe one split of a small historical dataset; individual
observations can materially affect the resulting metrics.

## Project structure

```text
CodeAlpha_Disease_Prediction/
├── models/
│   └── heart_disease_logistic_pipeline.joblib
├── examples/
│   ├── sample_input.csv
│   └── sample_predictions.csv
├── outputs/
│   └── .gitkeep
├── predict.py
├── metadata.json
├── requirements.txt
├── README.md
├── MODEL_CARD.md
├── REFERENCES.md
├── CITATION.cff
└── .gitignore
```

The Colab notebook documenting the full development workflow should
also be exported and added to the repository root.

## Installation

Use a compatible Python environment. The dependency versions used
during model development are listed in `requirements.txt`.

```bash
python -m venv .venv
python -m pip install -r requirements.txt
```

Activate `.venv` before running the following command.

## Run a prediction

```bash
python predict.py --input examples/sample_input.csv
```

By default, predictions are saved to `outputs/predictions.csv`.
```

```
To choose a different destination:

```bash
python predict.py --input examples/sample_input.csv --output outputs/custom_predictions.csv
```

The input CSV must contain all 13 original feature columns described
in `metadata.json`. Column order does not matter; the script aligns
the data to the recorded feature schema.

The prediction output retains the input columns and adds:

- `predicted_label`: class 0 or class 1 at the model's default threshold.
- `predicted_class`: human-readable description of the dataset target mapping.
- `class_1_model_score`: the model's estimated probability for class 1.

The score has not been established as a calibrated clinical risk.
Predictions must not be used for diagnosis or treatment decisions.

## Model interpretation

`permutation_importance.csv` records exploratory permutation
importance on the held-out test data. Importance indicates how
predictive performance changes when a feature is shuffled; it does
not demonstrate causation or clinical importance.

The test split was also used for this exploratory analysis. Its
importance rankings were not used to select features, tune parameters
or change the decision threshold.

## Reproducibility

- Dataset source and target mapping: `metadata.json`
- Feature schema and preprocessing groups: `metadata.json`
- Selected parameters and cross-validation score: `metadata.json`
- Dependency versions: `requirements.txt`
- Evaluation reports: CSV files in the repository root

Scikit-learn's `joblib` persistence format requires a compatible
environment. Only load model files from trusted sources.

## Limitations

- The dataset is small and historical.
- Only one held-out split was used for final evaluation.
- No independent external validation or prospective study has been performed.
- Performance may differ in other populations and settings.
- Permutation importance is exploratory and not causal.
- Neither accuracy nor class-1 scores establish clinical reliability.

## References

1. UCI Machine Learning Repository — Heart Disease:
   https://archive.ics.uci.edu/dataset/45/heart+disease
2. Scikit-learn — Pipelines and composite estimators:
   https://scikit-learn.org/stable/modules/compose.html
3. Scikit-learn — Cross-validation:
   https://scikit-learn.org/stable/modules/cross_validation.html
4. Scikit-learn — Permutation feature importance:
   https://scikit-learn.org/stable/modules/permutation_importance.html
5. Scikit-learn — Model persistence:
   https://scikit-learn.org/stable/model_persistence.html
6. Collins GS, Moons KGM, Dhiman P, et al. TRIPOD+AI statement:
   updated guidance for reporting clinical prediction models using
   regression or machine learning methods. BMJ. 2024;385:e078378.
   https://doi.org/10.1136/bmj-2023-078378
"""

(repo_dir / "README.md").write_text(readme, encoding="utf-8")

# ------------------------------------------------
# 4. Ignore generated outputs and runtime files
# ------------------------------------------------

(repo_dir / ".gitignore").write_text(
    "__pycache__/\n"
    "*.py[cod]\n"
    ".ipynb_checkpoints/\n"
    ".venv/\n"
    "venv/\n"
    ".env\n"
    ".DS_Store\n"
```